

# Aprendizado de Máquina Supervisionado –IMD3002

## Aula 21 – Redes Neurais Artificias Prática

João Carlos Xavier Júnior

[jcxavier@imd.ufrn.br](mailto:jcxavier@imd.ufrn.br)

# Multilayer Perceptron

## Utilizando Multilayer Perceptron (scikit-learn):

### 1.17.1. Multi-layer Perceptron

**Multi-layer Perceptron (MLP)** is a supervised learning algorithm that learns a function  $f(\cdot) : \mathbb{R}^m \rightarrow \mathbb{R}^o$  by training on a dataset, where  $m$  is the number of dimensions for input and  $o$  is the number of dimensions for output. Given a set of features  $X = x_1, x_2, \dots, x_m$  and a target  $y$ , it can learn a non-linear function approximator for either classification or regression. It is different from logistic regression, in that between the input and the output layer, there can be one or more non-linear layers, called hidden layers. Figure 1 shows a one hidden layer MLP with scalar output.

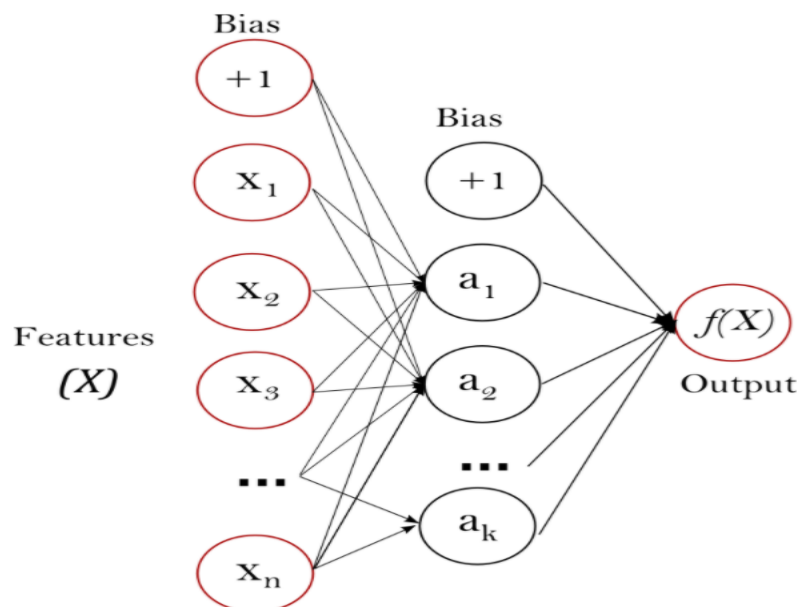


Figure 1 : One hidden layer MLP.

# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

`sklearn.neural_network.MLPClassifier`

```
class sklearn.neural_network.MLPClassifier(hidden_layer_sizes=100, activation='relu', *, solver='adam', alpha=0.0001,  
batch_size='auto', learning_rate='constant', learning_rate_init=0.001, power_t=0.5, max_iter=200, shuffle=True,  
random_state=None, tol=0.0001, verbose=False, warm_start=False, momentum=0.9, nesterovs_momentum=True,  
early_stopping=False, validation_fraction=0.1, beta_1=0.9, beta_2=0.999, epsilon=1e-08, n_iter_no_change=10, max_fun=15000)
```

[source]

[https://scikit-learn.org/stable/modules/generated/sklearn.neural\\_network.MLPClassifier.html](https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html)

# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

### Parameters:

**`hidden_layer_sizes : tuple, length = n_layers - 2, default=(100,)`**

The *i*th element represents the number of neurons in the *i*th hidden layer.

**`activation : {'identity', 'logistic', 'tanh', 'relu'}, default='relu'`**

Activation function for the hidden layer.

- 'identity', no-op activation, useful to implement linear bottleneck, returns  $f(x) = x$
- 'logistic', the logistic sigmoid function, returns  $f(x) = 1 / (1 + \exp(-x))$ .
- 'tanh', the hyperbolic tan function, returns  $f(x) = \tanh(x)$ .
- 'relu', the rectified linear unit function, returns  $f(x) = \max(0, x)$

**`solver : {'lbfgs', 'sgd', 'adam'}, default='adam'`**

The solver for weight optimization.

- 'lbfgs' is an optimizer in the family of quasi-Newton methods.
- 'sgd' refers to stochastic gradient descent.
- 'adam' refers to a stochastic gradient-based optimizer proposed by Kingma, Diederik, and Jimmy Ba

Note: The default solver 'adam' works pretty well on relatively large datasets (with thousands of training samples or more) in terms of both training time and validation score. For small datasets, however, 'lbfgs' can converge faster and perform better.

[https://scikit-learn.org/stable/modules/generated/sklearn.neural\\_network.MLPClassifier.html](https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html)

# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

```
## Carregando o algoritmo
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn.metrics import confusion_matrix

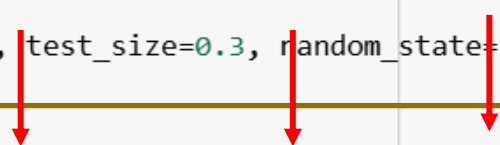
# Separando dataset em duas partes: treinamento e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=19)

### MLPClassifier e seus Hyperparametros
mlp = MLPClassifier(hidden_layer_sizes=(5), activation='relu', solver='adam', max_iter=500)
mlp.fit(X_train, y_train)

# Predict the response for test dataset
y_pred = mlp.predict(X_test)

# Model Accuracy, how often is the classifier correct?
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))

# Matriz de confusão
confusion_matrix(y_test, y_pred)
```



# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

```
⇒ Accuracy: 0.7291666666666666  
/usr/local/lib/python3.10/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:690: ConvergenceWarning:  
  warnings.warn(  
array([[77, 36],  
       [29, 98]])
```

ConvergenceWarning: Stochastic Optimizer: Maximum iterations (500) reached and the optimization hasn't converged yet.

```
### MLPClassifier e seus Hyperparametros  
mlp = MLPClassifier(hidden_layer_sizes=(5), activation='relu', solver='adam', max_iter=1000)  
mlp.fit(X_train, y_train)
```

```
⇒ Accuracy: 0.7125  
array([[90, 23],  
       [46, 81]])
```

# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

```
▶ ### MLPClassifier e seus Hyperparametros
mlp = MLPClassifier(hidden_layer_sizes=(5), activation='tanh', solver='adam', max_iter=1000)
mlp.fit(X_train,y_train)

#Predict the response for test dataset
y_pred = mlp.predict(X_test)

# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

# Matriz de confusão
confusion_matrix(y_test, y_pred)
```

```
⇒ Accuracy: 0.7125
array([[80, 33],
       [36, 91]])
```

# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

```
▶ ### MLPClassifier e seus Hyperparametros
mlp = MLPClassifier(hidden_layer_sizes=(5), activation='logistic', solver='adam', max_iter=1000)
mlp.fit(X_train,y_train)

#Predict the response for test dataset
y_pred = mlp.predict(X_test)

# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

# Matriz de confusão
confusion_matrix(y_test, y_pred)
```

```
⇒ Accuracy: 0.75
/usr/local/lib/python3.10/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:690: Conve
warnings.warn(
array([[85, 28],
       [32, 95]])
```



# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

```
▶ ### MLPClassifier e seus Hyperparametros
mlp = MLPClassifier(hidden_layer_sizes=(5), activation='identity', solver='adam', max_iter=1000)
mlp.fit(X_train,y_train)

#Predict the response for test dataset
y_pred = mlp.predict(X_test)

# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

# Matriz de confusão
confusion_matrix(y_test, y_pred)
```

```
⇒ Accuracy: 0.7083333333333334
array([[79, 34],
       [36, 91]])
```

# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

```
### MLPClassifier e seus Hyperparametros
mlp = MLPClassifier(hidden_layer_sizes=(5), activation='identity', solver='sgd', max_iter=1000)
mlp.fit(X_train,y_train)

#Predict the response for test dataset
y_pred = mlp.predict(X_test)

# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

# Matriz de confusão
confusion_matrix(y_test, y_pred)
```

⇒ Accuracy: 0.7583333333333333  
/usr/local/lib/python3.10/dist-packages/sklearn/neural\_network/\_multilayer\_perceptron.py:690: ConvergenceWarning:

```
array([[88, 25],
       [33, 94]])
```

# Multilayer Perceptron

## □ Parâmetro:

- ❖ hiddenLayers: número ( $n$ ) de camadas escondidas, podendo ser:
  - Qualquer valor inteiro maior ou igual a 1(\*).
  - Zero (sem hidden layers).
  - Valores nominais (a, i, o, t):
    - $a = (\text{attrs.} + \text{classes})/2$ ;
    - $i = \text{attrs.}$ ;
    - $o = \text{classes}$ ;
    - $t = \text{attrs.} + \text{classes}$ .

# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

```
## Carregando o dataset
dataset = pd.read_csv('HOG_128_16x16.csv', encoding='utf-8')
## dataset = pd.read_csv('CNN_VGG16_128_avg.csv', encoding='utf-8')
```

| BOS                  | BOT                 | BOU                 | BOV                  | ↓ BOW                |
|----------------------|---------------------|---------------------|----------------------|----------------------|
| hog_feature_1760     | hog_feature_1761    | hog_feature_1762    | hog_feature_1763     | hog_feature_1764     |
| 0.23223324193454983  | 0.18644244346586622 | 0.09129528073738279 | 0.05070440213743781  | 0.04324325267175837  |
| 0.5148234563068435   | 0.11363919113187013 | 0.17793841273788497 | 0.0410635650343945   | 0.023171527146640617 |
| 0.1649351572804906   | 0.08121711264475046 | 0.09787101947321612 | 0.132656090118366    | 0.19716954733012304  |
| 0.1649981948044552   | 0.14663396441633508 | 0.09760117655672286 | 0.0552581516369563   | 0.21465905077546138  |
| 0.024941557215741064 | 0.0269260131165977  | 0.02613001136554059 | 0.043954314782001916 | 0.17246264108413534  |
| 0.20926105349714255  | 0.08506431770928158 | 0.12757587281686217 | 0.19385884905006268  | 0.17598563008615722  |

- $1764 + 2 = 1766 / 2 = 883$
- 1764
- 2
- 1766

|     |     |                     |                      |
|-----|-----|---------------------|----------------------|
| 196 | cat | 0.21513768116614948 | 0.050562627063011525 |
| 197 | cat | 0.47513086366888907 | 0.11125784157089869  |
| 198 | cat | 0.1409249285124216  | 0.021920523120379287 |
| 199 | cat | 0.07544076427067736 | 0.0441268476007568   |
| 200 | cat | 0.07434725090269864 | 0.06463263258991343  |
| 201 | dog | 0.09185675914535085 | 0.07854568641341206  |
| 202 | dog | 0.14423293652915198 | 0.0749753975055662   |
| 203 | dog | 0.10787722793534482 | 0.010810385852481199 |
| 204 | dog | 0.31829991851841766 | 0.23026764973232733  |

# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

```
## Utilizando uma métrica de tempo
import time

### MLPClassifier e seus Hyperparametros
mlp = MLPClassifier(hidden_layer_sizes=(100), activation='relu', solver='adam', max_iter=1000)

start_time = time.time()
mlp.fit(X_train,y_train)
end_time = time.time()

#Predict the response for test dataset
y_pred = mlp.predict(X_test)

# Tempo total de treinamento
execution_time = end_time - start_time
print(f"Tempo de execução para treinamento: {execution_time:.4f} segundos")

# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

# Matriz de confusão
confusion_matrix(y_test, y_pred)
```

```
Tempo de execução para treinamento: 15.6500 segundos
Accuracy: 0.7333333333333333
array([[84, 29],
       [35, 92]])
```

# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

```
## Utilizando uma métrica de tempo
import time

### MLPClassifier e seus Hyperparametros
mlp = MLPClassifier(hidden_layer_sizes=(50, 30), activation='relu', solver='adam', max_iter=1000)

start_time = time.time()
mlp.fit(X_train,y_train)
end_time = time.time()

#Predict the response for test dataset
y_pred = mlp.predict(X_test)

# Tempo total de treinamento
execution_time = end_time - start_time
print(f"Tempo de execução para treinamento: {execution_time:.4f} segundos")

# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

# Matriz de confusão
confusion_matrix(y_test, y_pred)
```

```
Tempo de execução para treinamento: 10.8143 segundos
Accuracy: 0.7333333333333333
array([[85, 28],
       [36, 91]])
```

# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

**learning\_rate** : {'constant', 'invscaling', 'adaptive'} **default='constant'**

Learning rate schedule for weight updates.

- 'constant' is a constant learning rate given by 'learning\_rate\_init'.
- 'invscaling' gradually decreases the learning rate at each time step 't' using an inverse scaling exponent of 'power\_t'.  $\text{effective\_learning\_rate} = \text{learning\_rate\_init} / \text{pow}(t, \text{power\_t})$
- 'adaptive' keeps the learning rate constant to 'learning\_rate\_init' as long as training loss keeps decreasing. Each time two consecutive epochs fail to decrease training loss by at least tol, or fail to increase validation score by at least tol if 'early\_stopping' is on, the current learning rate is divided by 5.

Only used when `solver='sgd'`.

**learning\_rate\_init** : **float, default=0.001**

The initial learning rate used. It controls the step-size in updating the weights. Only used when solver='sgd' or 'adam'.

# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

```
## Utilizando uma métrica de tempo
import time

### MLPClassifier e seus Hyperparametros
mlp = MLPClassifier(hidden_layer_sizes=(50, 30), activation='relu',
                    solver='adam', learning_rate_init=0.0001, max_iter=1000)

start_time = time.time()
mlp.fit(X_train,y_train)
end_time = time.time()

#Predict the response for test dataset
y_pred = mlp.predict(X_test)

# Tempo total de treinamento
execution_time = end_time - start_time
print(f"Tempo de execução para treinamento: {execution_time:.4f} segundos")

# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

# Matriz de confusão
confusion_matrix(y_test, y_pred)

Tempo de execução para treinamento: 24.3440 segundos
Accuracy: 0.7458333333333333
array([[87, 26],
       [35, 92]])
```





# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

```
## Utilizando uma métrica de tempo
import time

#### MLPClassifier e seus Hyperparametros
mlp = MLPClassifier(hidden_layer_sizes=(50, 30), activation='relu',
                    solver='adam', learning_rate_init=0.01, max_iter=1000)

start_time = time.time()
mlp.fit(X_train,y_train)
end_time = time.time()

#Predict the response for test dataset
y_pred = mlp.predict(X_test)

# Tempo total de treinamento
execution_time = end_time - start_time
print(f"Tempo de execução para treinamento: {execution_time:.4f} segundos")

# Model Accuracy, how often is the classifier correct?
print("Accuracy:",metrics.accuracy_score(y_test, y_pred))

# Matriz de confusão
confusion_matrix(y_test, y_pred)

Tempo de execução para treinamento: 2.9521 segundos
Accuracy: 0.6541666666666667
array([[97, 16],
       [67, 60]])
```

# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

```
### GridSearch
from sklearn.model_selection import GridSearchCV

## Utilizando uma métrica de tempo
import time

# Definir o modelo
mlp = MLPClassifier(random_state=42)

# Definir os hiperparâmetros a serem testados
param_grid = {
    'hidden_layer_sizes': [(50), (100), (50, 50), (100, 50)],
    'activation': ['identity', 'logistic', 'tanh', 'relu'],
    'solver': ['sgd', 'adam'],
    'learning_rate_init': [0.001, 0.01, 0.1],
    'max_iter': [500, 1000]
}

# Configurar o GridSearchCV
grid_search = GridSearchCV(
    estimator=mlp,
    param_grid=param_grid,
    scoring='accuracy', # Pode ser alterado para outra métrica (ex.: 'f1', 'roc_auc')
    cv=3,               # Validação cruzada com 3 folds
    verbose=2,          # Para exibir detalhes durante a execução
    n_jobs=-1           # Usa todos os núcleos disponíveis
)
```

# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

```
start_time = time.time()

# Executar a busca
grid_search.fit(X_train,y_train)

# Melhor combinação de parâmetros
print("Melhores parâmetros encontrados:", grid_search.best_params_)

# Melhor pontuação obtida
print("Melhor pontuação:", grid_search.best_score_)

end_time = time.time()
# Tempo total de treinamento
execution_time = end_time - start_time
print(f"Tempo de execução para treinamento: {execution_time:.4f} segundos")

# Avaliar o modelo no conjunto de teste
best_model = grid_search.best_estimator_
test_score = best_model.score(X_test, y_test)
print("Acurácia no conjunto de teste:", test_score)
```

# Multilayer Perceptron

## ❑ Utilizando Multilayer Perceptron (scikit-learn):

Fitting 3 folds for each of 192 candidates, totalling 576 fits

Melhores parâmetros encontrados: {'activation': 'tanh', 'hidden\_layer\_sizes': 50, 'learning\_rate\_init': 0.001, 'max\_iter': 1000, 'solver': 'sgd'}

Melhor pontuação: 0.7656354819542676

Tempo de execução para treinamento: 3059.0322 segundos

Acurácia no conjunto de teste: 0.7541666666666667

/usr/local/lib/python3.10/dist-packages/sklearn/neural\_network/\_multilayer\_perceptron.py:690: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached  
warnings.warn(

# Dúvidas ...



# Obrigado!!!

